Information and Software Technology 140 (2021) 106686

Contents lists available at ScienceDirect

Information and Software Technology

journal homepage: www.elsevier.com/locate/infsof

Prioritizing code documentation effort: Can we do it simpler but better?

Shiran Liu, Zhaoqiang Guo, Yanhui Li *, Hongmin Lu, Lin Chen, Lei Xu, Yuming Zhou *, Baowen Xu

State Key Laboratory for Novel Software Technology, Nanjing University, China

ARTICLE INFO

Keywords:
Code documentation
Program comprehension
PageRank
Metrics

ABSTRACT

Context: . Due to time or economic pressures, code developers are often unable to write documents for all modules in a project. Recently, a supervised artificial neural network (ANN) approach is proposed to prioritize documentation effort "to ensure that sections of code important to program comprehension are thoroughly explained".

Objective: . However, as a supervised approach, there is a need to use labeled training data to train the prediction model, which may not easy to obtain in practice. Furthermore, it is unclear whether the ANN approach is generalizable, as it is only evaluated on several small data sets collected from API libraries.

Method: . In this paper, we propose an unsupervised approach based on improved PageRank to prioritize documentation effort. This approach identifies "important" modules only based on the dependence relationships between modules in a project. As a result, the PageRank approach does not need any training data to build the prediction model.

Results: . In order to evaluate the effectiveness of the PageRank approach, we use six additional large data sets collected from two larger libraries and four applications to conduct the experiment. The experimental results show that the PageRank approach is superior to the state-of-the-art ANN approach.

Conclusion: . Due to the simplicity and effectiveness, we advocate that the PageRank approach should be used as an easy-to-implement baseline in future research on documentation effort prioritization, and any newly proposed approach should be compared with it to demonstrate its effectiveness.

1. Introduction

Code documentations play an important role in software quality assurance [1–4,23–28]. High quality code documentation is helpful for program comprehension [29], test case generation [30], API Recommendation [31], fault localization [32], bug detection [33], and program repair [34]. However, many studies [5–8] point out that, due to time and monetary pressures, developers are often unable to write documents for all modules in a project. As a result, they may write incomplete documentation or neglect writing documentation entirely.

Many recent studies [9–14] proposed automatic source code summarization to mitigate manual documentation effort, which automatically generates natural language summaries of software systems by extracting important information from code. However, McBurney et al.'s study [15] shows that the document quality of the state-of-the-art source code summarization approach is still lower than that of manually-written expert summaries in many aspects.

In order to reduce the workload of developers in documentation effort, McBurney et al. [16] recently explored the possibility of using static source code metrics and textual analysis of source code to automatically prioritize documentation effort. The purpose of this is to "prioritize their documentation effort to ensure that sections of code important to program comprehension are thoroughly explained", i.e. enable that precious effort can be paid on writing documents for modules important for program comprehension. Based on three types of metrics, i.e. Static source Code Metrics (SCM), Textual Comparison Metrics (TCM), and Vector Space Model (VSM) [17], they adopted artificial neural network (ANN) [18] to build different supervised prediction models to classify modules (i.e. classes) in a software system into two categories: "important" and "non-important". The "important" modules refer to the modules that should be first documented by programmers. Based on an empirical study of five projects, they reported SCM was poor predictors whereas TCM and VSM were good predictors in documentation effort prioritization.

* Corresponding authors.
E-mail addresses: yanhuili@nju.edu.cn (Y. Li), zhouyuming@nju.edu.cn (Y. Zhou).

https://doi.org/10.1016/j.infsof.2021.106686
Received 20 July 2020; Received in revised form 25 June 2021; Accepted 7 July 2021
Available online 13 July 2021
0950-5849/© 2021 Elsevier B.V. All rights reserved.

However, McBurney et al. stressed that their study was "preliminary in nature" [16], as their ANN approach may be not "generalizable" and "mature" enough due to the existence of many threats to their study. First, they only used several small data sets from API libraries to conduct the experiment, although their original intention was to provide a general approach for prioritizing documentation effort. As a result, they pointed out that one important threat to the validity of their study is "using different API libraries, or using Java programs other than API libraries, may result in different conclusions" [16]. This means that it is unclear whether their approach has a good generalizability. Second, for a supervised approach, there is a need to use sufficient labeled training data to train the prediction model, which may be difficult to obtain in practice. This is especially true for a new type of projects or projects with little historical data collected. Third, their ANN approach not only has a high computation cost but also involves many parameters needed to be carefully tuned. This imposes substantial barriers to apply them in practice, especially for large projects. Fourth, their ANN approach is not compared against any baseline approaches. Consequently, it is unclear whether it is worth applying such a complex supervised approach in practice.

In the light of the above-mentioned facts, we aim to extend McBurney et al.'s research. In this study, we attempt to address the following problem: can we do simple but better in prioritizing documentation effort? Specifically, our motivation is three-fold. First, for the experiments of McBurney et al., we will perform external replication on their data sets. External replication plays a key role in empirical software engineering in enabling the software engineering community to understand experimental results (or conclusions) hold under which conditions [19–21]. Second, we want to extend their work: on the one hand, we use larger data sets from different API libraries to examine the generalizability of their ANN approach; on the other hand, we use applications to examine the generalizability of their ANN approach. Third, from the viewpoint of practical use, we expect that there is a simple yet effective approach for practitioners. To this end, we choose to apply an improved PageRank [22] to documentation effort prioritization. Unlike a supervised approach, the PageRank approach does not need any training data or many different types of metrics to build the prediction model. Indeed, the PageRank approach identifies "important" modules only based on the dependence relationships between modules in a target software system. Furthermore, the PageRank approach has a low computation cost and is easy to implement, which is scalable to large software systems. Our opinion is that, if the PageRank approach has a competitive effectiveness compared with the state-of-the-art ANN approach, it should be more practical for practitioners to use a simple approach rather than a complex approach. It should be noted that the introduction of PageRank to code documentation effort prioritization is also a good attempt to investigate the generalizability of the PageRank approach. In the current software engineering literature [35–40], PageRank is mainly used to identify important modules in an application, rather than in an API library. Note that, there are different views on whether a module is important in an application and in an API library. For an application, application developers can judge whether a module is important for understanding the application. However, the main purpose of an API library is to provide functions or interfaces for other programmers who are not the developers of the API library. Therefore, for a library, library users can also judge whether a module is important for understanding the library based on their experience in using the library. Furthermore, the inter-module dependences in an API library may be quite different from those in an application. Consequently, it is unclear that the PageRank approach used for applications (as reported in the current software engineering literature) is still suitable for API libraries (in terms of code documentation effort prioritization). In this sense, whether for the ANN or the PageRank, it is necessary to perform a comparative study between 2 algorithms across projects that are either applications or libraries to better explore the effectiveness of each approach.

Based on the above motivations, the core research question of this paper is: how effective is our PageRank approach in document effort prioritization, especially when compared with the state-of-the-art ANN approaches? To answer this question, on the one hand, we use the same data sets from open-source projects as used in McBurney et al.'s study [16] to conduct the experiments. On the other hand, we use six additional large data sets collected from open-source projects, including two API libraries and four applications, to conduct the experiments. Based on the nine data sets, we perform an extensive comparison between our PageRank approach and McBurney et al.'s ANN approach. The experimental results show that the PageRank approach is superior to the state-of-the-art ANN approach.

The main contributions of this paper are as follows:

1 We propose using an improved PageRank to prioritize important modules for code documentation effort. Being an unsupervised approach, the PageRank approach is training-free and is easy to implement in practice.
2 We conduct a comprehensive experiment comparison between the state-of-the-art and the PageRank approach. We are surprising to find that the PageRank approach has a competitive or even better performance, regardless of whether non-effort-aware or effort-aware performance indicators are considered.
3 We provide a replication package[1], including the data sets and scripts, which allows for replication and extension of our study and can also be used for other studies on quality assurance.

Our work has important implications. On the one hand, our work provides for practitioners an easy-to-use approach to prioritizing code documentation effort. On the other hand, our work contributes for researchers a simple yet strong baseline in code documentation effort prioritization. In the future, if a new prioritization approach is proposed, it should be compared against the PageRank approach to demonstrate its practical value. This will help our community develop more effective approaches for code documentation effort prioritization. In particular, our work demonstrates an important software engineering scenario that a simple solution could work well compared with complex solutions.

The rest of this paper is organized as follows. Section 2 introduces our research background. Section 3 presents the proposed PageRank approach. Section 4 describes the experimental designs. Section 5 reports the experimental results. Section 6 discloses the reason that the PageRank approach works well in document effort prioritization. Section 7 analyzes the threats to validity of our study. Section 8 concludes the paper and outlines the future work.

2. Background

In this section, we introduce our research background, including code document effort prioritization and the advantages and disadvantages of supervised vs. unsupervised prediction approaches.

2.1. Code documentation effort prioritization

Code documentation is a key component of software quality assurance. Good code documentation can greatly promote the understanding of software system, accelerate the process of learning and reusing code, increase developer productivity, simplify maintenance, and therefore improve the reliability of software [23–28]. In contrast, poor code documentation is one of the main reasons for the rapid deterioration of software system quality [26]. Therefore, code documentation is an irreplaceable necessity to enhance software reliability. In the online

[1] https://github.com/sticeran/PrioritizingDocumentationEffort

appendix A[2], we summarize the fundamental role of code documentation in many areas of software quality assurance.

However, in real software development, due to time and economic pressure (such as rapid iteration and release of software development), it is often difficult for developers to write (high-quality) code documents for all modules [5–8]. Furthermore, not all code always needs to be documented to a high quality level. Therefore, to save developers' precious time, McBurney et al. [16] recently proposed the concept of code documentation effort prioritization, which refers to "Programmers must prioritize their documentation effort. The sections of code that are the most important for developers to understand should be documented first." Therefore, the role of code documentation effort prioritization is to help developers of a software project allocate time reasonably and improve cost-effectiveness: write code documents for important modules within a limited time to "ensure that sections of code important to program comprehension are thoroughly explained" [16].

Since the term "code documentation effort prioritization" derives from McBurney et al.'s work, we elaborate on their approach here. They proposed a supervised artificial neural network (ANN) approach to automatically prioritizing important modules for documentation effort. They first collected the actual priority labels of classes, then collected several types of code metrics, and finally compared supervised models trained by ANN. At the first step, to get the actual priority of classes, they collected user scores (i.e. expert experience) on five Java projects at the class-level and divided the classes in each project into two categories: "important" and "non-important". Users rated a class based on their perception of its importance. In their research, the top 25% of classes scored by users were marked as "important", while the rest were marked as "non-important". At the second step, to investigate the effectiveness of different types of code metrics in predicting documentation effort priorities, they collected three types of code metrics (SCM, TCM, VSM). In the online appendix[2], appendix D lists all metrics of SCM and TCM used in their study [16]. On a given project, they took the following measures to collect the corresponding metrics. For SCM, they analyzed source code to collect size metrics, complexity metrics, and object-oriented metrics. For TCM, they compared source code and the corresponding project homepage to compute their textual similarity as well as binary variables indicating whether class/package names appeared at the same time. For VSM, they collected tf/idf (term frequency/inverse document frequency) for each word in the source code. At the last step, they used artificial neural networks (ANN) to build various supervised prediction models and performed a 10-fold cross validation for evaluating their effectiveness. Based on the experimental results, they found that VSM were the best predicators, while SCM were the worst predicators.

Note that McBurney et al. did not distinguish the project types when they proposed the ANN approach. That is to say, it was expected that the ANN approach would work for both applications and libraries. However, in their experiment, McBurney et al. chose only several small libraries to conduct their experiment. The reason was that it was easy for the participants to rate the importance of modules in libraries (i.e. it was easy to obtain labeled data from libraries), thus facilitating build the ANN models. In [16], however, McBurney et al. highlighted that this was an important validity to their ANN approach. In this study, we use six more projects to conduct the experiment, including two larger libraries and four applications. This will help analyze the practical usefulness of McBurney et al.'s ANN approach.

2.2. Supervised vs. unsupervised approaches

A supervised approach aims to learn a function, from labeled data (i.e. a number of instances with features and labels), to model the relationship between features and the corresponding labels observable in the data. Artificial neural network (ANN) is one of the most commonly used supervised techniques. Building a supervised model often requires a large amount of labeled data. In contrast, an unsupervised approach aims to learn a distribution, from unlabeled data (i.e. a number of instances with features), to discover and present the interesting structure in the data.

Compared with an unsupervised approach, a supervised approach leverages prior knowledge in the training data and hence is often expected to have a higher prediction effectiveness. However, the disadvantages are three-fold. First, the labeled training data may incur a significant data collection cost or even are very difficult to obtain. Second, the training data and the testing data may have different distributions such that the prior knowledge from the training data may not be well applied to the testing data. Third, it is often computation-intensive to train a supervised model, especially for complex supervised modeling techniques such as ANN.

In our prior work, we found that, in defect prediction field, simple unsupervised models had a competitive or even superior prediction effectiveness compared with the existing supervised models in the literature [51]. Inspired by this work, we want to explore whether the same phenomena can be observed in documentation effort prioritization. Specifically, we want to explore whether PageRank [22] can be used to prioritize "important classes" for documentation effort. PageRank is a well-known algorithm that measures the importance of web pages. Given a set of hyperlinked set of web pages, the PageRank algorithm abstracts it as a graph, in which nodes correspond to web pages and edges correspond to hyperlinks. In this graph, a node is believed as important if it has many in-edges or has in-edges from important nodes. In other words, it considers both the number and quality of hyperlinks to a page when determining the importance of the page. In nature, PageRank is an unsupervised approach, as labeled training data are not needed. In the last decade, PageRank has been found many applications in software engineering [35–40]. If we find an unsupervised approach such as PageRank has a competitive or even superior prediction effectiveness to the-state-of-the-art supervised approach, this will lead to a large benefit for practitioners in documentation effort prioritization.

3. Approach

This section introduces our PageRank approach to documentation effort prioritization. First, at a high level, we outline the process of identifying important classes. Second, we give the definitions of dependence relationships used in our study. Third, we introduce how to compute the importance scores of classes by applying a weighted PageRank algorithm. Note that, in this study, a module can be a class or an interface in an object-oriented software system.

3.1. Approach overview

For a given target project, our PageRank approach calculates the importance score of each module to complete the documentation effort prioritization. Specifically, the importance score calculation consists of the following three steps. First, build an inter-module dependence graph by analyzing the dependences between modules in the target software. Second, calculate the importance score of each module according to an extended PageRank algorithm. Third, sort the importance scores in descending order and classify modules whose importance scores exceed a threshold (e.g. top 5%) as important modules. These important modules are the ones that should be documented first.

We next use an example shown in Fig. 1 to illustrate our approach. As can be seen, the example target software system consists of four modules: three classes (i.e. `A`, `B`, and `D`) and one interface (i.e. `C`). In Fig. 1, we use annotations (dependence type and modules that are depended on) to explicitly describe the dependence relationships of each class with other classes. For example, for the second line of code in class `A`: "`public D d`

[2] https://github.com/sticeran/PrioritizingDocumentationEffort Appendix-Prioritizing code documentation effort Can we do it simpler but better.pdf

```
public class A extends B implements C{//CI: B,C

    public D d = new D();//CA: D
    public C c = new D();//CA: C
    int var_A1 = d.var1_D;
    int var_A2 = D.var2_D_static;

    public A(D d1)//CM: D
    {
        d1.method_D();//MM: D
        D.method_D_static();//MM: D
    }

    public C Method_A(B b)//CM: B,C
    {
        D d2 = new D();
        d2.method_D();//MM: D
        return c;
    }

    public void method_C(B b) {}//CM: B

}
```

```
public class B {

    public void method_B(A a, D d) {}//CM: A,D

}
```

```
public interface C {

    public void method_C(B b);//CM: B

}
```

```
public class D implements C{//CI: C

    public int var1_D = 1;
    public static int var2_D_static = 2;

    public void method_D() {}

    public static void method_D_static() {}

    public void method_C(B b) {}//CM: B

}
```

Fig. 1. An example object-oriented software system consisting of four modules.

= new D ()", we can see that the type of attribute d of class A is class D. Therefore, class A and class D have a Class-Attribute dependence relationship (i.e. CA, class A depends on class D). In order to describe this fact, we add a comment "//CA: D" to the end of this line. In our study, we take into account the following four types of dependences among modules:

(1) Class-Inheritance dependence (CI): One class (interface) is a subclass (interface) of another class; one class implements an interface in which the implemented interface is regarded as the parent class and the class implementing the interface is regarded as the subclass.
(2) Class-Attribute dependence (CA): The type of a class's (interface's) attribute is another class (interface).
(3) Class-Method dependence (CM): The parameter type or return value type of a class's method is another class (interface); the parameter type of an interface's method is another class (interface).
(4) Method-Method dependence (MM): A method in one class calls another class's method.

As stated in [35], the above four types of dependences are the main inter-module dependences in an object-oriented software system. Note that these four types of dependences can be efficiently obtained by a static analysis.

Fig. 2 shows the calculation process of our PageRank approach. As can be seen, a dependence graph is constructed based on the static analysis of source code. The dependence graph is a directed weighted graph. In this graph, each node represents a module, the direction of each edge represents the dependence direction, and the weight on each edge represents the total times of a module depends on another module. For example, the total times of class A depending on class B is 3 (one CI: B and two CM: B). Therefore, the weight on the edge <A, B> is 3. Based on the dependence graph, we use a weighted PageRank algorithm (see Section 3.3 for detail) to calculate the importance score of each module. The higher the importance score is, the more important the module is. According to the score ranking, the top k% modules are regarded as important modules. These important modules are the ones that developers should document first. For example, if k% = 25%, only B is in the top 25%. Therefore, B is recognized as an important module for documentation effort.

3.2. Generating edge-weighted module dependence graph

Given a target software system, by static analysis, we can easily extract the following dependence relationships among modules: Class-Inheritance dependence (CI), Class-Attribute dependence (CA), Class-Method dependence (CM), and Method-Method dependence (MM).

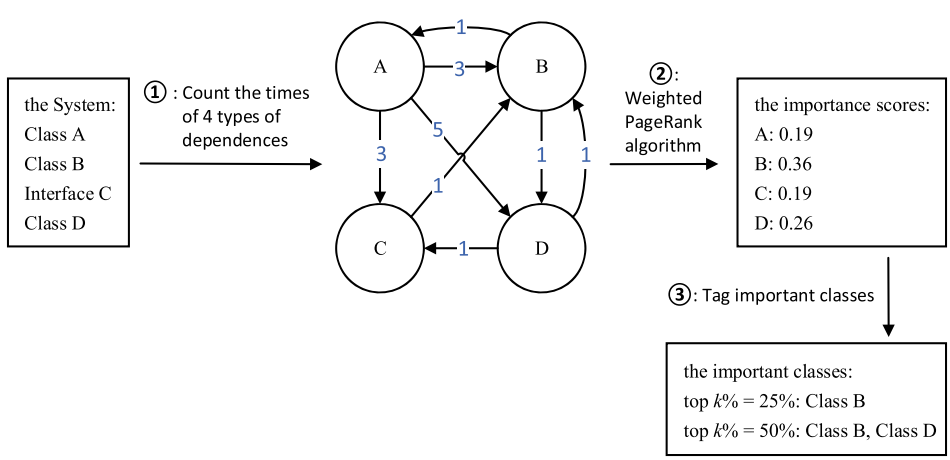

Fig. 2. The calculation process of our PageRank approach.

Based on these dependences, we generate a directed weighted graph to depict the relationships among modules.

For the simplicity of presentation, in the following, we formalize the types of dependences and dependence graph. For a given object-oriented software system S, let C(S) be the set of all modules (non-inner classes or interfaces) in S. Furthermore, let |C(S)| be the number of elements in C (S).

Definition 1. Inter-module dependence. Given a software system S, for u, v∈C(S) and u ≠ v, we have the following definitions:

(1) CI(u, v) denotes the number of inheritance dependence between u and v. If v is the parent of u, the value is 1, otherwise it is 0.
(2) CA(u, v) denotes the number of class-attribute dependences between u and v, which is equal to the number of attributes' type of v in u.
(3) CM(u, v) denotes the number of class-method dependences between u and v, which is equal to the number of methods in u with v as the parameter type or return value type.
(4) MM(u, v) denotes the number of method-method dependences between u and v, which is equal to the number of times of methods in v is called in u.

Definition 2. Edge-weighted module dependence graph. Let (u, v) denote one edge and W(u, v) denote the weight of one edge. For a software system S, the corresponding dependence graph is defined as G_S = (N, E, W), where

- N = C(S)
- E = {(u, v) | u, v∈N ∧ u ≠ v ∧ W(u, v) > 0}
- W = {W(u, v) | W(u, v) = CI(u, v) + CA(u, v) + CM(u, v) + MM(u, v)}.

Definition 3. In-Edge and Out-Edge. For any node u in a dependence graph G_S = (N, E, W), its In-Edge and Out-Edge are defined as:

- In-Edgeu = {(v, u) | v ∈N ∧ v ≠ u ∧ W(v, u) > 0}
- Out-Edgeu = {(u, v) | v ∈N ∧ u ≠ v ∧ W(u, v) > 0}

3.3. Deriving importance score by improved PageRank-VOL

Consistent with the original PageRank algorithm's criterion [22] for identifying the importance of web pages, we identify important modules according to the following criterion: a module is important if and only if many important modules depend on it. This criterion recursively defines whether a module is important depends on two factors: (1) the more modules depend on it, the more important it is; (2) the more important modules depend on it, the more important it is. Inconsistent with the original PageRank algorithm, we further take into account the third factor: the strength of dependence among modules. In other words, the higher strengths depend on it, the more important it is. The reason is due to the following intuition (or practical experience with API libraries). For a user who has no prior knowledge of a system, when he starts to understand the system randomly from one module, the dependencies contained in a module itself provide a bridge to jump to other modules. The more times that a module is dependent on another module are, the greater the probability that the user will jump to another module to continue reading is. Taking Fig. 1 as an example, because module `D` is relied on more times in module `A` (that is, more code lines in module `A` involve module `D`), users are more likely to jump to module `D` to understand the function of module `A`.

To this end, we use the improved PageRank based on Visits Of Links (PageRank-VOL) proposed by Kumar et al. [41] to calculate importance scores of modules. The reason why we choose PageRank-VOL is that the idea of this method is consistent with the idea of edge-weighted dependence graph proposed by us. The PageRank-VOL has considered the number of outgoing and inbound links between web pages and it assigns more rank value to the outgoing links which is most visited by users. In this manner, a page rank value is calculated based on visits of inbound links. Indeed, PageRank-VOL is a special case of edge-weighted PageRank [42].

The original calculation formula of PageRank-VOL is:

$$PR(u) = d \sum_{v \in B\left(u\right)} \frac{L_u \; PR(v)}{TL(v)} + (1-d) \tag{1}$$

Based on the above equation, we change the "(1-d)" part to "(1-d)/m". Consequently, we have the PageRank-IVOL equation:

$$PR(u) = d \sum_{v \in B\left(u\right)} \frac{L_u \; PR(v)}{TL(v)} + \frac{(1-d)}{m} \tag{2}$$

The objective of this goal is to ensure that the sum of the importance score over all the nodes in an edge-weighted module dependence graph equals to 1. The notations in our importance score equation are:

- d is a dampening factor and it is generally taken 0.85.
- m is the total number of nodes (i.e. modules).
- u and v (u ≠ v) represent one node respectively.
- B(u) is the set of nodes that point to u.
- PR(u) and PR(v) are important scores of node u and v respectively,
- Lu is the weight of the In-Edgeu which points to node u from v
- TL(v) is the sum of weight of all Out-Edgev which point to other nodes from v

In our implementation, we use the iterative method to solve PageRank-IVOL. The termination condition is that the error is less than 10^{-7} or the number of iterations reaches 100.

To explain the working of PageRank-IVOL, let us take the dependence graph in Fig. 2 as an example. The PageRank for nodes A, B, C, and D are calculated as follows:

$$PR\left(A\right) = d\left(PR\left(B\right) * 1/2) + \left(1-d\right)/4$$

$$PR\left(B\right) = d\left(PR\left(A\right) * 3/11 + PR\left(C\right) + PR\left(D\right) * 1/2) + \left(1-d\right)/4$$

$$PR\left(C\right) = d\left(PR\left(A\right) * 3/11 + PR\left(D\right) * 1/2) + \left(1-d\right)/4$$

$$PR\left(D\right) = d\left(PR\left(A\right) * 5/11 + PR\left(B\right) * 1/2) + \left(1-d\right)/4$$

Let P_0 be the initial column vector whose sum of column elements is 1, where each element represents the initial importance score for each node. Furthermore, let e be a column vector in which each element is 1. M is the transition probability matrix. Then, we have:

$$P_0 = \begin{bmatrix} PR_0(A) \\ PR_0(B) \\ PR_0(C) \\ PR_0(D) \end{bmatrix}$$

$$P_1 = d \cdot M \cdot P_0 + \frac{1-d}{4} \cdot e$$

$$P_1 = d\cdot\begin{bmatrix} 0 & \frac{1}{2} & 0 & 0 \\ \frac{3}{11} & 0 & 1 & \frac{1}{2} \\ \frac{3}{11} & 0 & 0 & \frac{1}{2} \\ \frac{5}{11} & \frac{1}{2} & 0 & 0 \end{bmatrix}\cdot P_0 + \frac{1-d}{4}\cdot\begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}$$

$$P_n = d\cdot M\cdot P_{n-1} + \frac{1-d}{4}\cdot e$$

Generally speaking, the initial important scores are the same, so $PR_0(A) = PR_0(B) = PR_0(C) = PR_0(D) = 0.25$. The iteration of the above equation is continued until the termination condition is reached (e.g., the error is less than 10^{-7} or the number of iterations reaches 100). As a result: PR(A) = 0.19, PR(B) = 0.36, PR(C) = 0.19, and PR(D) = 0.26.

4. Experimental design

In this section, we introduce the experimental design to answer our core research question (i.e., investigate the effectiveness of our proposed PageRank approach in prioritizing code documentation effort), including the research framework, baseline approach, data sets, performance indicators, and evaluation method.

4.1. Research framework

Our overall research framework is shown in Fig. 3. On the one hand, for a given target project, we use an improved PageRank to compute module importance score based on the inter-module dependence graph generated by static source code analysis. On the other hand, we strictly follow the description in [16] to implement the ANN approach. Specifically, we implement two types of ANN models: the SCM-ANN model built with static source code metrics and the VSM-ANN model built with vector space model metrics. After that, the same test data set is used to compare the predictive effectiveness of the PageRank-IVOL approach and the ANN approach, including F1, recall, and precision.

Our core purpose is to investigate whether the PageRank approach is better than the recently proposed ANN approach in code documentation effort prioritization. If the PageRank approach shows a superior performance, it would be better for practitioners to use it in practice due to its simplicity and the unsupervised nature.

4.2. Baseline approach

We use McBurney et al.'s metrics-based ANN approaches [16] as the baseline approaches to evaluate the effectiveness of our PageRank approach. In [16], McBurney et al. used the following four group metrics to build the ANN prediction models: (1) only the static source code metrics (SCM); (2) only the text comparison metrics (TCM); (3) only the VSM metrics; and (4) SCM + TCM. In our study, we use their SCM-based ANN model and the VSM-based ANN model as the baselines to investigate the effectiveness of our PageRank approach. Note that we do not include the ANN models built with TCM or SCM + TCM as the baselines. The reasons are three-fold. First, they were inferior to the VSM-based ANN model according to the experimental results in McBurney et al.'s study. Second, it is not uncommon that the homepage of a project does not evolve with the project versions consistently, thus leading to the information mismatch between the homepage and source code. That is to say, we cannot guarantee that the content of the current project homepage corresponds to the version of the project we use. Third, what is more important, they are not applicable if a project does not have a corresponding home page. In contrast, both the SCM-based and VSM-based ANN model are based on only the source code of a project and hence are easier to be applied in practice.

We use the same ANN classifier and settings as used in McBurney et al.'s study [16] to build the ANN models. Specifically, the used classifier is the multilayer neural network library in the Neuroph[3] framework with the following settings: two hidden layers, the maximum number of iterations of 1000 runs, the learning rate is 0.25, the number of hidden nodes is 10, and a feed-forward fully connected architecture. Considering the large number of metrics in SCM and VSM, we generate four types of supervised ANN models: SCM model (static code metrics without feature selection), VSM model (VSM metrics without feature selection), SCM_FS model (static code metrics with feature selection), and VSM_FS model (VSM metrics with feature selection). In our study, we use CFS [43] (the Correlation-based Feature Selection) to build the SCM_FS and VSM_FS models, by which the number of features reserved can be automatically determined.

Note that, in McBurney et al.'s study [16], the feature selection technique they used for SCM model is Chi-Squared test. In our study, we use CFS rather than Chi-Squared test to conduct the feature selection. The reasons are two-fold. First, Chi-Squared test needs to set the number of reserved features in advance. However, it was not reported in McBurney et al.'s study [16]. In contrast, CFS is the most commonly used feature selection technique that can automatically determine the reserved features according to characteristics of the data set. Second, using different feature selection techniques can provide a new perspective for experiments. In McBurney et al.'s study [16], the effectiveness of SCM model with Chi-Squared test was not as good as VSM model. We can examine whether using CFS for SCM model can improve the effectiveness of SCM. The potential threat of different feature selection techniques to our experiments will be discussed in Section 7.2.

Furthermore, in McBurney et al.'s study [16], they did not apply feature selection to the VSM tf/idf metrics, i.e. the VSM_FS model was not considered. However, for a project, the VSM metrics often consist of thousands of metrics. This means that the number of features is far larger than the number of instances in a data set. Given this satiation, it will be very likely to produce an overfitted model. In this case, the resulting model may not generalize well to unseen data. Therefore, in our study, we take into account the VSM_FS model.

4.3. Data sets

Our data sets are divided into two parts: Old data sets and New data sets, as shown in Table 1(a) and (b). The old data sets are from McBurney et al.'s study [16], while the new data sets are from Sora's study [37]. Specifically, the subject projects in the old data sets consist of three open-source Java API libraries (i.e., NanoXML 2.2.1, JExcelAPI 2.6.12, and JGraphT 0.9.1). The subject projects in the new data sets consist of two open-source Java API libraries (i.e., Ant 1.6.1[4] and JHotDraw 6.0b.1[5]) and four Java applications (i.e., ArgoUML 0.9.5, jEdit 5.1.0, JMeter 2.0.1, and Wro4j 1.6.3). In each data set, one instance corresponds to a class and consists of two parts: a number of metrics (i.e. SCM and VSM) and a label indicating whether it is important or not.

The purposes of adding new data sets are two-fold. On the one hand, we want to investigate whether their ANN approach can be generalized to other API libraries. On the other hand, we want to investigate how the ANN approach performs in applications (i.e. software projects that are not API libraries). In [16], McBurney et al. stressed that it was an important threat to the validity of their study whether their ANN approach can be generalized to other libraries or Java applications. With these new data sets, we can evaluate the generalizability of McBurney et al.'s ANN approach.

In our study, we first download the source code for the projects used

[3] http://neuroph.sourceforge.net/

[4] https://ant.apache.org/. "Apache Ant is **a Java library** …".

[5] https://sourceforge.net/projects/jhotdraw/. "This project provides **a library** of easy to use 2D graphics editor tools ……".

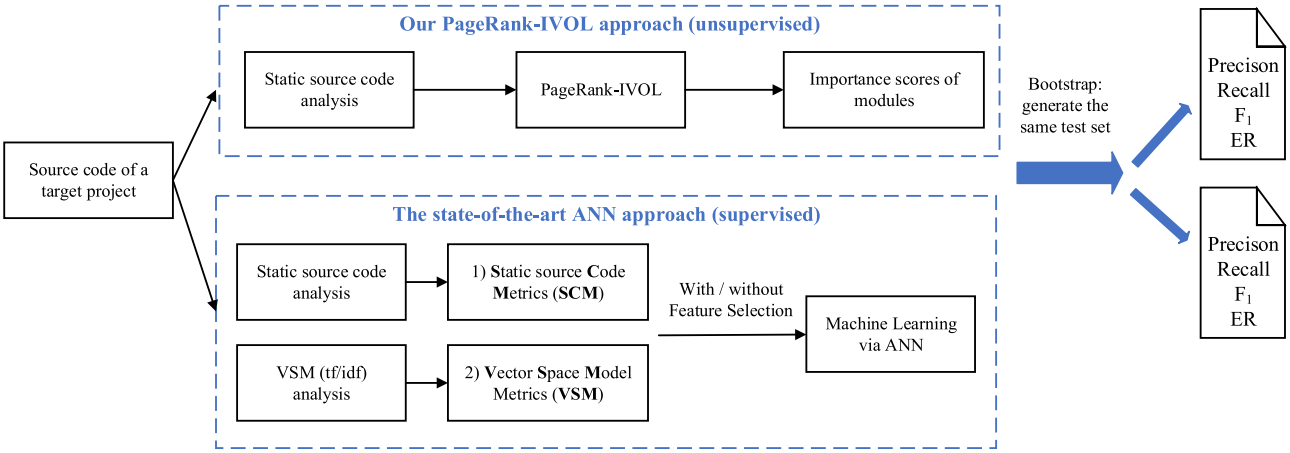

Fig. 3. Overall research framework in this paper.

Table 1
Data sets used in our study.

(a) Old data sets

Project	Version	Type	The total number of classes	The total number of classes used in experiment [1]	Important classes (top 25%)	Source of Category labels
NanoXML	nanoxml-2.2.1	Java API library	28	21	6	[16]
JExcelAPI	jexcelapi-2.6.12	Java API library	458	50	11	
JGraphT	jgrapht-0.9.1	Java API library	194	176	47	

1. In the Old data sets, the number of classes used in the experiment is equal to the number of classes scored by users.

(b) New data sets

Project	Version	Type	The total number of classes	The total number of classes used in experiment	The number of important classes	Source of Category labels
Ant	ant-1.6.1	Java API library	664	664	8	[37]
JHotDraw	jhotdraw-6.0b.1	Java API library	498	498	9	
ArgoUML	argouml-0.9.5	Java application	823	823	12	
jEdit	jedit-5.1.0	Java application	539	539	7	
JMeter	jmeter-2.0.1	Java application	224	224	13	
Wro4j	wro4j-1.6.3	Java application	357	357	12	

in McBurney et al.'s and Sora's studies. Then, we take the following measures to collect the data.

Label data collection. For each library in the old data sets, McBurney et al. [16] published the importance of the classes used in their experiments, including class name and the corresponding subjective score evaluated by graduate students based on their perception of importance. subjective score evaluated by graduate students who were using these API libraries (projects) for the first time, through a programming task, to understand the purpose of these API libraries. Based on the importance score, McBurney et al. [16] marked the top 25% classes as "important" and the rest as "non-important". For each library or application in the new data sets, Sora [37] published the names of important classes identified via the information from diverse sources, including the project tutorial, design description, development documents, and code analysis. As a result, for each class in the old and new data sets, we can easily obtain the corresponding label (i.e. "important" or "non-important").

Metric data collection. When collecting SCM, for each project, we first used a commercial tool Understand[6] to generate the corresponding udb database by parsing its source code. The udb database stored the entities and their relationships in the project. Then, we develop a Perl script to collect SCM based on the udb database. When collecting the VSM metrics, similar to McBurney et al. [16], we first developed a Python script to remove all special characters and numbers. Then, we generated the VSM metrics via Weka's StringToWordVector filter.

From Table 1(a), we can see that, for each library in the old data sets, not all the classes are used in McBurney et al.'s study. On a given project, McBurney et al. invited three graduate students to score classes according to their importance for document with respect to program comprehension. Before scoring, each participant was asked to do a programming task involving the library (at most 70 minutes). Then, the participant used the entire remainder of the 90-minute time limit to score the importance to document each method within a class (the order of the methods was randomized). After, each class was assigned "the average of the average scores for each scored method in the class" [16]. Due to the 90-minute time limit, not all the classes in a library were scored by the participants. Consequently, the identified important classes, obtained from the scored classes in a library, may not be the real important classes for the whole library. This problem is especially true for NanoXML 2.2.1 and JExcelAPI 2.6.12: 75% classes in NanoXML 2.2.1

[6] http://www.scitools.com

were scored while only 11% classes were scored in JExcelAPI 2.6.12. Nonetheless, in our study, we keep the old data sets for our experiments. The purpose of this is to enable a direct comparison of our PageRank approach with the ANN approach reported in their study.

From Table 1(b), we can see that, for each project (library or application) in the new data sets, all the classes in the whole project were involved to identify the important classes [37]. Furthermore, in order to obtain accurate label data, diverse information sources, mainly provided by the original developers of the project, are used to label important classes in Sora's study [37]. For Ant, the important classes were provided in its project tutorial and development document. For JHotDraw, the important classes were provided in its development document and complemented by code analysis. For ArgoUML, the important classes were provided in its detailed architectural descriptions. For jEdit, the important classes were pointed out by its development document. For JMeter, the important classes were mentioned in its design document. For Wro4j, the important classes were provided in its design overview. As can be seen, compared with the old data sets, the new data sets have the following characteristics: (1) the important classes are provided by the original developers or official documents rather than several students; (2) the important classes are identified on the basis of the whole project rather than a part of the project; and (3) the number of instances for each project is large enough to draw statistically meaningful conclusions.

As can be seen, the label data collection approaches are not the same for the old and new data sets. In the old data sets, the label data were determined by the users of the subject projects. In the new data sets, the label data were actually determined by the developers of the subject projects. However, at a high level, they serve the same purpose: indicating whether a module is important for programmers (e.g. software maintainers for maintaining an application or library users for using an API library) to understand a project, regardless of whether the project is an application or a library. Indeed, this diverse label data collection approaches provides a very good opportunity to draw robust conclusions from the comparison between the PageRank and ANN approaches.

4.4. Performance indicator

According to [16], in nature, documentation effort prioritization is a classification problem: the modules are classified into either "important" or "non-important" modules. Therefore, the following indicators can be used to evaluate the effectiveness of a prediction approach m:

$$precision = \frac{TP}{TP + FP}$$

$$recall = \frac{TP}{TP + FN}$$

$$F_1 = \frac{2 \times precision \times recall}{precision + recall}$$

Here, TP refers to the number of actual important modules that are predicted as important, FP refers to the number of actual non-important modules that are predicted as important, and FN refers to the number of actual important modules that are predicted as non-important.

In practice, for a target project, it is very likely that only a very few modules are the truly important modules. Consequently, the prediction approach m may have a low precision even if the recall is high. If developers document all the predicted important modules, many falsely important modules will also be documented. This is no problem if the time/economic resource is enough. However, if the time/economic resource is not enough, i.e. not all the predicted important modules can be documented, it is wise to filter out falsely important modules (before documenting modules) by inspecting the predicted important modules. In this case, there is a strong need to evaluate the cost-benefit of m, which is ignored in McBurney et al.'s study [16]. Given a prediction approach, how to evaluate its cost-benefit in helping identify the truly important modules from the predicted important modules? Without loss of generalization, for a target project, assume that there are n modules in total, of which k module are truly important. For a prediction approach m, assume that, of all the x predicted important modules, y modules are truly important. In order to evaluate the cost-benefit of the prediction approach m, we make the following assumptions

- All the x predicted important modules are inspected;
- The inspection is perfect, i.e. all the y truly important modules in the inspected x modules are found;
- For computing a benefit, we use the random selection model (modules to be inspected are selected from the n modules by chance) as a baseline of comparison.

As a result, when using the prediction approach m, the percentage of modules needed to be inspected is x/n × 100%, while the percentage of truly important modules found is y/k × 100%. Similar to [44,45], we evaluate how effective the prediction model m is in reducing the effort for inspection compared to a random model that achieves the same recall of finding truly important modules. To this end, we use the following ER (effort reduction) indicator:

$$ER = \frac{effort(random) - effort\left(m\right)}{effort\left(random\right)}$$

Here, effort(m) = x/n × 100% and effort(random) = y/k × 100%. In other words, we have:

$$ER = \frac{\frac{y}{k} - \frac{x}{n}}{\frac{y}{k}}$$

A positive ER indicates that the prediction approach m is superior to a random model. A higher ER indicates a better cost-benefit.

By the above-mentioned four indicators (i.e. precision, recall, F_1, and ER), we can comprehensively evaluate the effectiveness of a prediction approach in documentation effort prioritization. In practice, the ER indicator is especially important when there is no enough time/economic resource to document all the predicted important modules.

4.5. Evaluation method

For each class in the test data, both the PageRank and ANN approaches will output a value ranging between 0 and 1, which can be explained as the predicted importance. As a result, the classes in the test data can be ranked in descending order according to the predicted importance. Given such a ranking list, developers can determine a threshold k% to classify the classes into two groups: the top k% as "important" and the bottom (100-k)% as "non-important" (in practice, developers can flexibly select a threshold according to their workload). In this context, we can compute the precision, recall, F_1, and ER indicators for both the PageRank and ANN approaches to compare their effectiveness. Similar to McBurney et al. [16], we vary k% from 5% to 50% with a step length of 5% to conduct a comprehensive comparison between the PageRank and ANN approaches. Generally speaking, if an approach has higher precision, recall, F_1, and ER values under small thresholds, it is better in documentation effort prioritization.

For each data, we use the 100-times out-of-sample bootstrap [46] technique to conduct the experiment. According to [47], the out-of-sample bootstrap is the least biased model validation technique in terms of both threshold-dependent and threshold-independent performance measures. Assuming the size of the data set is N, an out-of-sample bootstrap uses the sampling with replacement to extract N instances as the training data and use the un-sampled instances as the test data. In our experiment, we run out-of-sample bootstrap 100 times. The seed used to generate random numbers in out-of-sample bootstrap is set as

follows: 0 at the beginning and 1 is added for each time.

- For the ANN approach, at each run, the SCM and VSM models are trained using the training data and tested on the test data. After that, we use the predicted value to rank classes in descending order. Then, the top k% classes are selected as the predicted important classes.
- For the PageRank approach, we compute the importance for each class based on the classes in the data set (rather than in the whole project) and their dependence relationships. For the old data sets, this means that only the scored classes in McBurney et al.'s study are used. For example, for the jexcelapi-2.6.12 data sets, only 50 classes (rather than 458 classes) and their dependence relationships are analyzed to build the edge-weighted inter-module dependence graph.

For both the PageRank and ANN approaches, the final effectiveness is the average of the results on 100-times out-of-sample bootstrap, regardless of whether the precision, recall, F_1, or ER is taken into account.

5. Experimental results

In this section, we report the experimental results of our core research question. In order to facilitate readers to understand and better present the results, we present the results in two parts, RQ1 and RQ2.

5.1. RQ1: how effective is the PageRank approach in identifying "important" classes compared with the state-of-the-art ANN approaches?

In this section, we first give our three observations from the results, and give our analysis to each observation. More observations or analyses are put into RQ2 (a supplement to RQ1) for further elaboration and

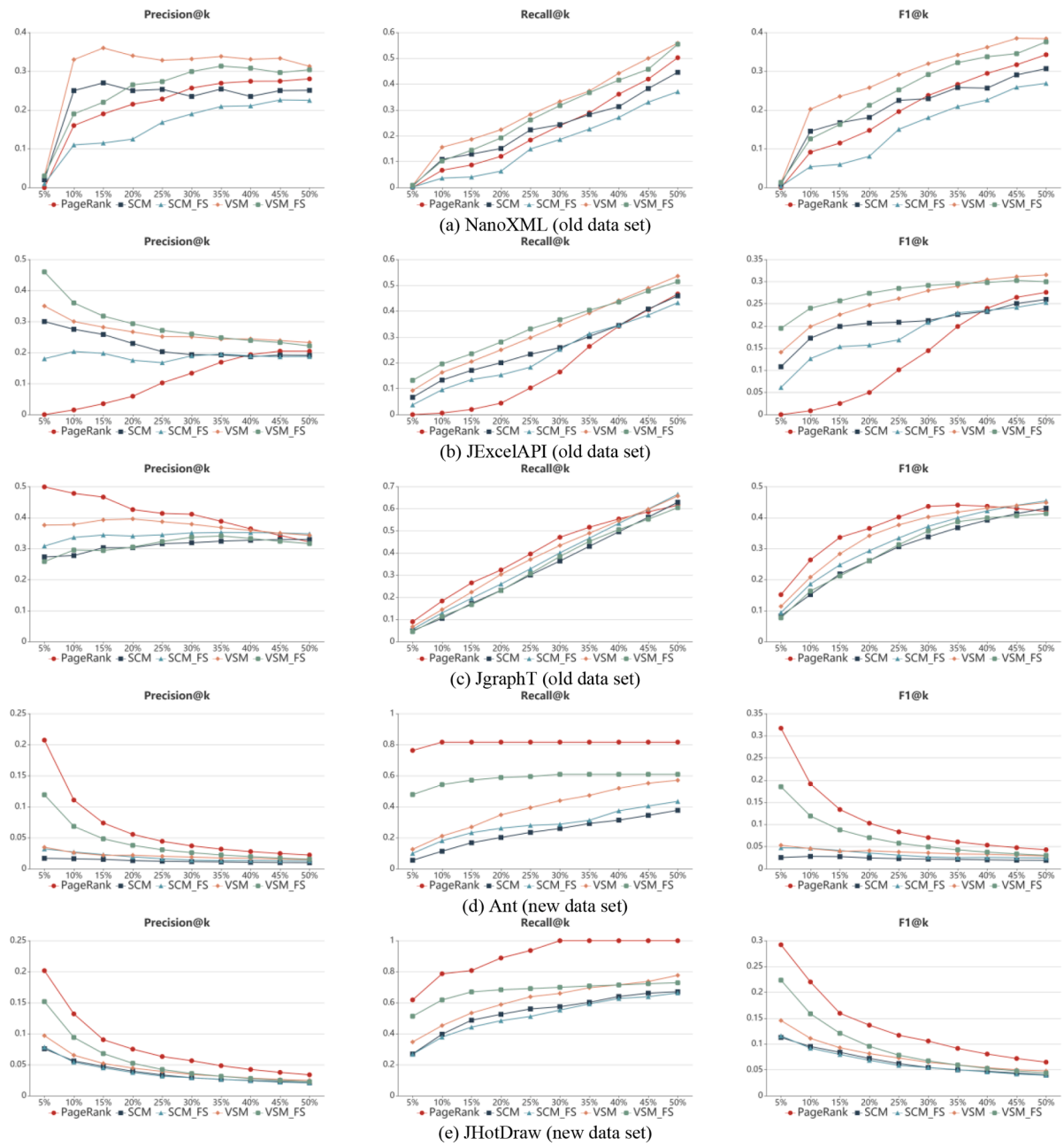

Fig. 4. Comparison of precision, recall and F_1 between the PageRank model and supervised models on the API libraries ("@k" denotes the corresponding results of the model when k takes different thresholds).

analysis.

Observations. From Fig. 4 (old data sets) and Fig. 5 (new data sets), we have the following observations: (1) the PageRank approach is superior to the ANN method on the larger libraries, while the PageRank approach is inferior to the ANN method on the smaller libraries; (2) the PageRank approach is clearly superior to all the ANN approaches on all applications (in new data sets); and (3) with the decrease of threshold k, the precision (or F_1) of the PageRank approach increase more obviously than the ANN approaches on most projects (libraries or applications).

Specifically, the first observation is as follows: On the JGraphT, Ant, and JhotDraw libraries, the PageRank approach is better than all the ANN approaches (SCM, SCM_FS, VSM, and VSM_FS); On the NanoXML library, the PageRank approach is between SCM (SCM_FS) and VSM (VSM_FS); On the JExcelAPI library, the PageRank approach is inferior to all the ANN approaches. The third observation is as follows: For the JGraphT library (in the old data sets), the precision of the PageRank approach increases while all the ANN approaches decreases with the decrease of threshold. For all libraries or applications in the new data sets, the precision and F_1 of the PageRank approach increases more significantly than all the ANN approaches with the decrease of threshold.

Analysis 1. For the observation (1), we believe that there are two possible reasons for the large performance difference. First, the PageRank approach only uses a part of classes in each library for the old data sets to calculate the importance score, which may lose many dependence relationships among classes reflecting the importance of classes. Combining with Table 1(a), it seems that the greater the difference between the number of classes used in the experiment and the total number of classes in a library, the worse the PageRank approach will be. We will discuss this point in appendix C.1. Second, in the old data sets, the importance of the scored classes in each library may not be accurate enough to represent the importance of all classes in the whole library. As we explained in Section 4, for a limited period of time, participants failed to rate all classes (including those they had seen but not shown in the random order in the questionnaire). In view of the fact that the total number of libraries in the old data sets and the number of scored classes in each library are relatively small, it is necessary to use more data sets to evaluate the effectiveness of the PageRank approach and the ANN approach.

Analysis 2. For the observation (2), we believe that the main reason is that **the PageRank approach can captures the essence of importance concept more successfully: the dependence relationships (especially the indirect dependences) among modules**, rather than their characteristics such as size, complexity, or tf/idf, play a key role in determining which modules are important. From the perspective of users of a software system, for a given module, if many other modules directly or indirectly depend on it, it is naturally important for understanding the whole software system, no matter whether the module is simple or complex. However, the code metrics on which the ANN approach is based are (probably) not highly correlated with the module importance. For the ANN approaches, the used metrics include source code metrics (size, complexity, object-oriented metrics, and comment ratio) and tf/idf. Of these metrics, except few object-oriented metrics such as DIT and NSC (see appendix D for the meaning of code metrics), the remaining metrics have no explicit connection to the direct dependence relationships. Furthermore, all the source code and tf/idf metrics cannot capture the indirect dependence relationships (the PageRank approach tackles this problem by a recursive way to capture the indirect dependences). Recently, Peitek et al. [56] demonstrated that the four types of code metrics (code size, vocabulary size, control flow complexity, and data flow complexity) had no correlation or only low to medium correlation for program understanding through neurobiological experiments (a functional magnetic resonance imaging (fMRI) study). To some extent, this can explain why ANN approaches are not effective. As a result, it is not surprised that the PageRank approach has an outstanding effectiveness in finding truly important classes compared with the state-of-the-art ANN approaches.

Another possible factor for observation (2) is that the label data in the new data set is imbalanced, while the ANN classifier cannot be well trained in the imbalanced data, which may make the ANN approach inferior to PageRank approach. In order to avoid the interference of the second possible factor, we have investigated the effectiveness of rebalanced ANN approaches in appendix C.3. The experimental results show that the PageRank approach is always the best or the second best (only the rebalanced ANN approach based on VSM outperforms the PageRank approach on some projects). Therefore, the PageRank approach does capture the essence of the module importance more successfully. Note that in the experiments of Section 5, we do not deal with the imbalanced data for three reasons. First, imbalanced data also appeared in the old data sets, while McBurney et al. [16] did not use data rebalancing technique. We follow the same experimental setup in order to examine the effectiveness of ANN approaches applied to different data sets under the same experimental setup. Second, Song et al. [52] found that the indiscriminate application of imbalanced learning technique can be harmful, so we do not rashly adopt imbalanced learning techniques (it is beyond the scope of this study to evaluate the effect of different imbalanced learning techniques on our experimental results). Third, the experimental results also can reflect the following experimental significance: ANN approaches are not suitable for this kind of imbalanced data sets (if no specific processing is done); while the unsupervised PageRank approach is not sensitive to imbalanced data and it can achieve a relatively high recall.

Analysis 3. For the PageRank approach, a rapid increase in precision with the decrease of the threshold indicates that a higher proportion of the top classes ranked by PageRank are truly important classes. In other words, the PageRank approach can give higher importance scores to truly important classes, so the truly important classes rank higher (except for the NanoXML and JExcelAPI projects). Therefore, the PageRank approach works better at a smaller threshold ($k\% \leq 25\%$). The main reason is that, under the PageRank approach, classes with rich inter-class dependencies are more likely to have larger importance scores. Intuitively, classes with rich dependencies can help programmers understand the functions of classes and the calling relationships of classes, so these classes may have a high degree of relevance to program understanding. Indeed, as analyzed in [37], classes that are truly important for program understanding are often rich in inter-class dependencies. Consequently, truly important classes tend to be ranked higher under the PageRank approach. For the ANN approaches, classes with larger size, complexity, or tf/idf metrics may have larger importance scores and hence tend to be ranked higher. However, these classes are not necessarily important for program understanding. This is the possible reason why the PageRank approach performs more outstandingly for a smaller threshold k% compared with the ANN approaches. Note that similar phenomena cannot be observed on the NanoXML and JExcelAPI projects. This may be due to the fact that a considerable proportion of classes and their related dependencies in these two projects are not taken into account when the PageRank approach computes the importance scores for classes. We will show this in appendix C.1.

5.2. RQ2: when only low thresholds are considered, how effective is the PageRank approach in identifying "important" classes compared with the state-of-the-art ANN approaches?

RQ2 is a supplement to RQ1. In RQ1, the setting of threshold (the top 5%∼50% classes are predicted as "important") is consistent with that of McBurney et al.' study [16]. In practice, it might make more sense to observe the effectiveness under smaller thresholds. Because for program comprehension, the meaning of important classes is that reading a limited number of important classes can help users without prior knowledge quickly understand the whole system. When the threshold is too high, it means that a method needs to identify most classes as important classes, which will make important classes lose their own

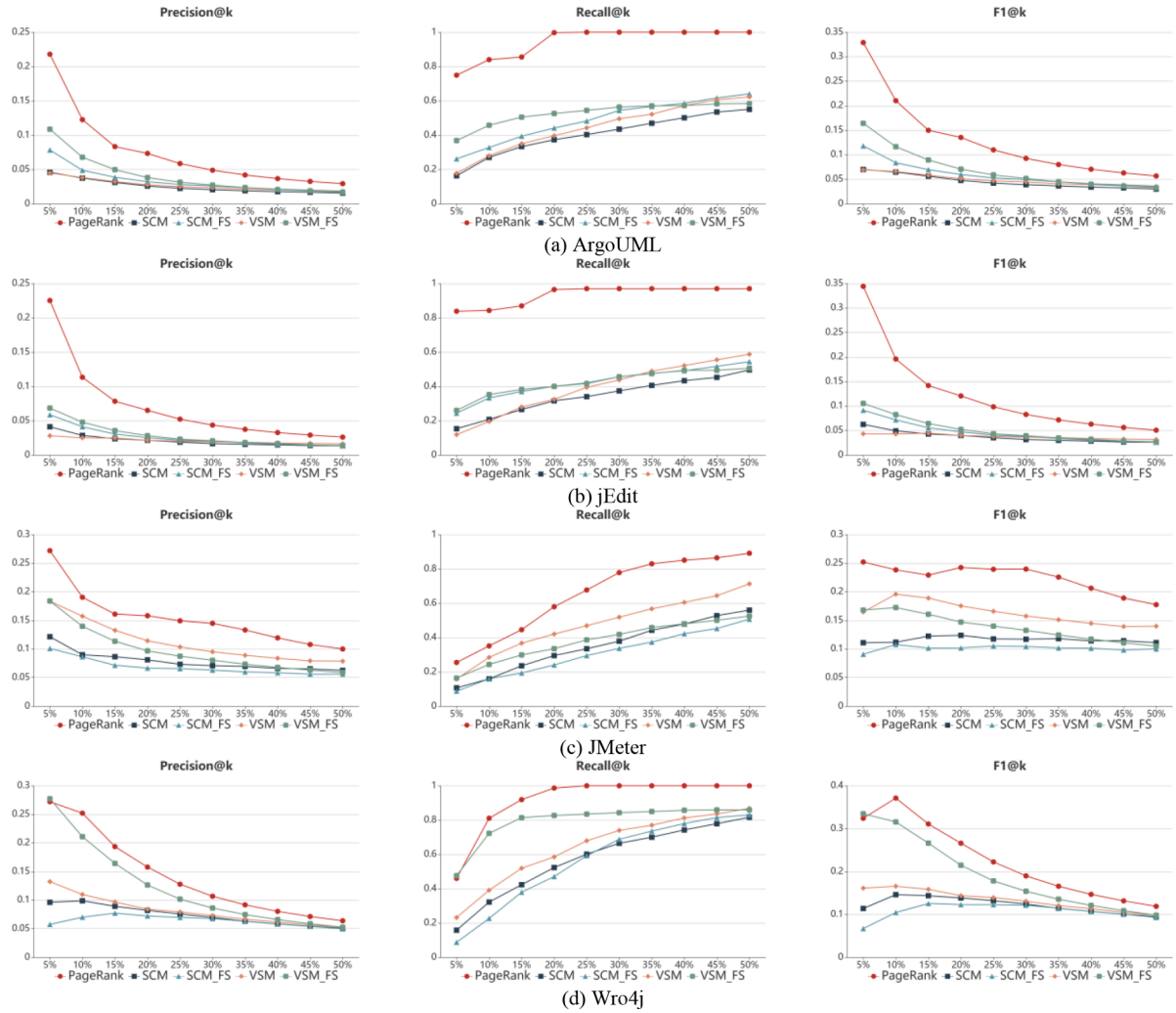

Fig. 5. Comparison of precision, recall and F_1 between the PageRank model and supervised models on applications in the new data sets ("@k" denotes the corresponding results of the model when k takes different thresholds).

meaning.

Table 2 report the results, including precision, recall, F_1, and ER, for the PageRank and ANN approaches under the threshold top 10% (the results under top 5% are similar, please see appendix B for detail). In each sub-table, for each project, we first compare the PageRank approach against each of the ANN approaches to obtain the following two values: the p-value via the Wilcoxon signed-rank test [48] and the effect size via Cliff's δ [49]. Then, we apply Benjamini-Hochberg's method [50] to adjust the above p-values to control false discovery. Finally, we mark the background of cells under the ANN approaches with different colors. A pink background means that the corresponding ANN approach is significantly worse than the PageRank approach (the adjusted p-value < 0.05). A blue background means that the corresponding ANN approach is significantly better (the adjusted p-value < 0.05). A white background means that the corresponding ANN approach is not significantly different from the PageRank approach (the adjusted p-value ≥ 0.05). In particular, we use "N", "S", "M", and "L" to respectively represent that the corresponding effect size is negligible ($|\delta| < 0.147$), small ($0.147 \leq |\delta| < 0.33$), moderate ($0.133 \leq |\delta| < 0.474$) and large ($|\delta| \geq 0.474$).

Observations. From Table 2(a)–(d), we have the following observations: (1) on most libraries or applications (except on the NanoXML and JExcelAPI libraries), the PageRank approach is significantly better than the ANN approaches in terms of precision, recall, and F_1, with a large effect size in most cases; (2) both PageRank approach and the ANN approaches have relatively low precision values (compared with other evaluation indicators); and (3) on most libraries or applications (except on the NanoXML and JExcelAPI libraries), the PageRank approach is much better than random approach (the reason is that the ER value is positive) and the ANN approaches according to the cost-benefit ER, with a large effect size in most cases. The ER values of PageRank on all projects are higher than 0.6, and most of them are higher than 0.8. However, the ER values of the ANN approaches are all not higher than 0.6 except the VSM_FS approach. The VSM_FS approach, which has the highest ER value among the ANN approaches, even has a negative ER value on one project.

Analysis 1. The results of observation (1) further indicate that the PageRank approach gives higher importance scores to truly important classes, so the truly important classes rank higher. Based on the performance on the libraries and applications, we can see that the effectiveness and generalizability of the ANN approaches are not as good as the PageRank approach. On the NanoXML library, the PageRank approach performs similarly to the ANN approaches. On the JExcelAPI library, the performance of the PageRank approach is worse than the ANN approaches. We believe that the possible reason is that the number of classes (i.e. the labeled classes) used for experiments is far less than the total number of classes in this library, which will lead to the loss of the dependencies among classes. As a result, the performance of the PageRank approach is suppressed (please see evidences shown in appendix C.1 for detail).

Table 2
Performance comparison.

(a) Precision under the top 10%					
Project	Precision				
	SCM	SCM_FS	VSM	VSM_FS	PageRank
NanoXML	0.25(N)	0.11(N)	0.33(S)	0.19(N)	0.16
JExcelAPI	0.28(M)	0.2(M)	0.3(L)	0.36(L)	0.02
JGraphT	0.28(L)	0.34(M)	0.38(S)	0.3(L)	0.48
Ant	0.02(L)	0.03(L)	0.03(L)	0.07(M)	0.11
JHotDraw	0.06(L)	0.05(L)	0.07(L)	0.09(M)	0.13
ArgoUML	0.04(L)	0.05(L)	0.04(L)	0.07(L)	0.12
jEdit	0.03(L)	0.04(L)	0.03(L)	0.05(L)	0.11
JMeter	0.09(L)	0.09(L)	0.16(S)	0.14(S)	0.19
Wro4j	0.1(L)	0.07(L)	0.11(L)	0.21(S)	0.25
(b) Recall under the top 10%					
Project	Recall				
	SCM	SCM_FS	VSM	VSM_FS	PageRank
NanoXML	0.11(N)	0.04(N)	0.16(S)	0.1(N)	0.07
JExcelAPI	0.13(L)	0.1(M)	0.16(L)	0.2(L)	0.01
JGraphT	0.11(L)	0.13(M)	0.14(M)	0.11(L)	0.18
Ant	0.11(L)	0.18(L)	0.21(L)	0.54(M)	0.82
JHotDraw	0.4(L)	0.38(L)	0.45(L)	0.62(S)	0.79
ArgoUML	0.27(L)	0.33(L)	0.28(L)	0.46(L)	0.84
jEdit	0.21(L)	0.33(L)	0.2(L)	0.35(L)	0.84
JMeter	0.16(L)	0.16(L)	0.28(S)	0.24(S)	0.35
Wro4j	0.32(L)	0.23(L)	0.39(L)	0.72(S)	0.81
(c) F_1 under the top 10%					
Project	F1				
	SCM	SCM_FS	VSM	VSM_FS	PageRank
NanoXML	0.15(N)	0.05(N)	0.2(S)	0.13(N)	0.09
JExcelAPI	0.17(L)	0.13(M)	0.2(L)	0.24(L)	0.01
JGraphT	0.15(L)	0.19(M)	0.21(M)	0.16(L)	0.26
Ant	0.03(L)	0.05(L)	0.05(L)	0.12(M)	0.19
JHotDraw	0.1(L)	0.09(L)	0.11(L)	0.16(M)	0.22
ArgoUML	0.06(L)	0.08(L)	0.07(L)	0.12(L)	0.21
jEdit	0.05(L)	0.07(L)	0.04(L)	0.08(L)	0.2
JMeter	0.11(L)	0.11(L)	0.2(S)	0.17(S)	0.24
Wro4j	0.15(L)	0.1(L)	0.17(L)	0.32(S)	0.37
(d) ER under the top 10%					
Project	ER				
	SCM	SCM_FS	VSM	VSM_FS	PageRank
NanoXML	0.13(N)	0.06(N)	0.17(N)	0.09(N)	0.09
JExcelAPI	0.26(L)	0.19(M)	0.3(L)	0.35(L)	0.01
JGraphT	-0.05(L)	0.07(M)	0.17(S)	-0.06(L)	0.33
Ant	0.18(L)	0.28(L)	0.34(L)	0.64(M)	0.83
JHotDraw	0.47(L)	0.47(L)	0.6(L)	0.72(S)	0.83
ArgoUML	0.42(L)	0.47(L)	0.47(L)	0.65(L)	0.87
jEdit	0.3(L)	0.44(L)	0.3(L)	0.49(L)	0.84
JMeter	0.32(L)	0.32(L)	0.52(S)	0.46(S)	0.61
Wro4j	0.53(L)	0.41(L)	0.59(L)	0.81(S)	0.86

Analysis 2. For the observation (2), we believe that the main reasons are two-folds. First, on the new data set, the number of labeled important classes accounts for a relatively low proportion of the total number of classes (e.g. for Ant, the number of important classes (8) accounts for 1.2% of the total number of classes (664)), while the threshold (10%) is relatively high. This results in the precision not being very high. Second, according to [16], the students in their experiment had not finished scoring all the classes in each library. Therefore, these labeled "important" classes may not be the truly important classes in the library for program comprehension, and these labeled "important" classes may also miss the truly important classes. This may lead to a reduction in precision.

According to the actual needs, moderately lowering the threshold may further improve the precision. However, we think that sometimes low precision is acceptable depending on the scenario and purpose. For example, in practice, recall may be more important to ensure the comprehension of team for the project. That is the most important classes needed to be documented as many as possible. At this point, a reduction in precision is tolerated, that is, some less important classes are also misidentified as important classes and are documented. At this point, there may be no loss to the comprehension of the project, but there may be some promotion significance.

Analysis 3. The results of observation (3) indicate that selecting important classes by the PageRank approach can indeed reduce the documentation effort that developers need to write for program comprehension compared with the random approach and the ANN approaches. That is, the PageRank approach does achieve the prioritization of documentation effort. It is not difficult to find that with the threshold rising, the ER value of ANN approaches may be negative. This suggests that from the cost-benefit (ER) point of view, the ANN approaches may even increase the documentation effort. Therefore, the PageRank approach is more competent for the prioritization of documentation effort.

Overall, the PageRank approach is superior to the-state-of-the-art supervised ANN approach for the prioritization of documentation effort.

6. Why does the PageRank approach work well?

The experimental results in Section 5 show that the PageRank approach is a good baseline. In this section, we present a further analysis to understand why the PageRank approach works well in documentation effort prioritization.

According to [57,58], module dependencies play a key role in understanding a software system. Ko et al. [58] studied how developers understood unfamiliar code through controlled experiments. They found that developers used direct and indirect dependencies obtained from static analysis to find important code to understand [58]: 58% of developers looked for relevant code through direct dependence (27% of them returned the code they had recently browsed); 42% of developers looked for relevant code through indirect dependence (34% of them returned the code they had recently browsed). That is, in general, developers need to find important code through multiple switching of module dependencies. In their study, direct dependence referred to the dependence relationships between modules by class declaration, variable declaration, method call, and so on, while indirect dependence referred to a dependence relationship with transitive nature generated through multiple direct dependencies.

In our research, in order to calculate the importance of modules in a software system, we first abstract the system into a weighted module dependence graph, where edges correspond to four kinds of dependence: CI, CA, CM and MM (see Section 3.2). Then, on this basis, we use the improved PageRank algorithm to quantify the importance of modules. According to [59–61], the PageRank algorithm can be understood as a Markov chain: (1) the states are pages, and the transitions are the links between pages; (2) a random surfer will follow the transitions to browse pages. If a sink page is reached, the surfer will pick another page at random to browse; and (3) the PageRank value of a page reflects the probability of staying on that page, i.e., the importance of that page. In the context of program understanding, it can be seen that the Markov chain naturally simulates the actual behavior of developers in looking for important code. We think that this is the reason why the PageRank approach can work well in identifying "important" modules in a software system. However, such a behavior of developers in looking for important code revealed by Ko et al.'s study in [58] cannot be well simulated by a simple approach (i.e., aggregating the dependencies and complexity of a module to quantify its importance) or by a supervised learning approach such as the ANN approach (i.e., weighting all the metrics of a module to quantify its importance).

From numerical calculation, the PageRank algorithm makes two basic assumptions when identifying "important" modules: (1) the more modules depend on it, the more important it is (expressed concisely as "**Quantity Criterion**"); (2) the more important modules depend on it, the more important it is (expressed concisely as "**Quality Criterion**"). The former involves direct dependence relationships between modules, while the latter involves indirect dependence relationships between modules. Specifically, in each iteration of the PageRank algorithm, the direct dependence relationships between modules constitute the transition probability matrix of the module state transition process (i.e.

Quantity Criterion). In the nth iteration, the PageRank scores of modules are obtained by multiplying the transition probability matrix with the PageRank scores of the $n\text{-}1^{th}$ iteration (i.e. Quality Criterion). Before reaching a stationary distribution, many state transitions are undergone, so the indirect dependencies (generated through multiple direct dependencies) also affect the importance of the modules. In this sense, the PageRank algorithm takes into account both direct and indirect dependences that play a role in program understanding.

We next use an example to demonstrate that the PageRank approach can make a better appropriate recommendation than the other approaches. To this end, we first introduce two baselines. The first baseline, called QNC, identifies important modules only by the QuaNtity Criterion in the PageRank. QNC does not iterate and only counts the total number of times that each module is depended on by other modules and ranks them accordingly. The importance score of each module is expressed in the following formula:

$$Score(v) = \sum_{v \in N} CI(u,\ v)\ +\ CA(u,\ v)\ +\ CM(u,\ v)\ +\ MM(u,\ v)$$

The second baseline, called ADC (Aggregating Dependencies and Couplings), identifies important modules only by aggregating all fan in (FanIn), fan out (FanOut), and coupling between modules (CBO). The FanIn, FanOut, and CBO are collected by the Understand tool, corresponding to "CountInput", "CountOutput", and "CountClassCoupled" metrics[7] respectively. The importance score of each module is expressed in the following formula:

$$Score(v) = FanIn(v) + FanOut(v) + CBO(v)$$

Since neither QNC nor ADC involves indirect dependences, these two baselines are suitable for observing the role of indirect dependences (the Quality Criterion in PageRank) in the identification of important modules.

Taking the Ant project as an example, Table 3 lists the truly important modules correctly identified by PageRank, QNC, and ADC, respectively. In Table 3, the 1st column lists all the truly important module names on the Ant project. The 2nd column is the ranking obtained by PageRank. The 3rd column is whether the truly important modules are identified as "important" when the top 5% of instances of the PageRank approach are predicted to be "important". The 4th column is the statistical results of QNC (i.e., the total number of times that each truly important module is depended on by other modules). The 5th column is the ranking obtained by QNC. The 6th column is the identification results of QNC for the truly important modules within the top 5% threshold. The 7th column is the statistical results of ADC (i.e., the number of couplings and dependencies contained in each truly important module). The 8th column is the ranking obtained by ADC. The 9th column is the identification results of ADC for the truly important modules within the top 5% threshold. Table 3 shows visually that, under the same threshold, PageRank makes more appropriate recommendations but QNC and ADC fail (PageRank correctly identified 8/9, while QNC only correctly identified 3/9 and ADC only correctly identified 3/9). In particular, "ProjectHelper2.java", "RuntimeConfigurable.java", and "UnknownElement.java" instances do not have rich direct dependencies, but they are still correctly identified by PageRank, while both QNC and ADC fail. This shows that it is not good enough to identify important modules only according to the Quantity Criterion or just aggregating all dependencies and couplings (that is, using only direct dependences does not work well). Therefore, indirect dependences (the Quality Criterion in PageRank) plays a crucial role in identifying important modules.

[7] The definition of "CountInput", "CountOutput" and "CountClassCoupled" metrics can be found on the official website of Understand: https://support.scitools.com/t/what-metrics-does-undertand-have/66

Further, Fig. 6 displays the module dependence graph of the Ant project. Due to the large number of modules in the Ant project (664), we only list modules in the top 5% of importance score obtained by the PageRank approach, of which the nodes marked in red are truly important modules (true positive). In Fig. 6, larger nodes indicate higher PageRank importance scores, and wider edges indicate greater weight of edges (see definition 2 in Section 3.3 for the concept of weight). Fig. 6 shows visually that, almost all of the truly important modules have rich direct and indirect dependences. The only exception is the "ProjectHelperImpl.java" module, which is depended on / called zero times by other modules, but is still considered important by developers (the source of instance labels). By reading the module code, we found that the "ProjectHelperImpl.java" module is the help documentation / instruction program for the project (see the comment statement "original helper." in the file header). Naturally, the "ProjectHelperImpl.java" module does not provide functional code, and therefore is not depended on / called by other modules, but it is still important for program understanding. Similar observations to those in Table 3 and Fig. 6 can also be observed on other projects (projects with more than 100 modules). Therefore, in most cases, what makes the PageRank approach better at identifying the truly important modules is that the PageRank algorithm can better capture the direct and indirect dependences between modules (because direct and indirect dependences play an important role in program understanding [57,58]).

Last but not least, it is important to note that the purpose of our study is not to deeply analyze the factors that affect documentation prioritization effort, but to find a simpler baseline to help future research better demonstrate promising methods for documentation prioritization. The deeper analysis of the effectiveness of these approaches (ANN and PageRank) can be referred to the previous literatures [18,22,59–61].

7. Threats to validity

In this section, we discuss the threats to construct validity, internal validity, and external validity.

7.1. Construct validity

Construct validity is the degree to which the independent and dependent variables accurately measure the concept they purport to measure.

In our study, the used dependent variable is a binary label that indicates whether a class is important. For the old data sets, we use the labels shared online by McBurney et al. [16]. These labels are set according to the importance scores they collected from graduate students. As mentioned in [16], for a given class, they used the average of the scores over methods in the class given by graduate students as its importance score. If a method was not scored, it was not considered when computing the corresponding class importance. Therefore, the labels on the old data sets may be inaccurate, either due to the subjective bias caused by graduate students or due to the missing scores of methods in a class. For the new data sets, according to the description in literature [37], they collected the labels via information from diverse sources, including the project tutorial, design description, and development documents (most from official documents). In this sense, the label formation on the new data sets should be accurate and hence the construct validity of the dependent variable should be considered satisfactory.

The independent variables used in this study are static code metrics, tf/idf metrics, and PageRank scores, each of which has a clear definition. As for the static code metrics and tf/idf metrics, we develop our own scripts (based on the mature tools Understand and Weka respectively) to collect the data. Note that, for the old data sets, we do not use McBurney et al.'s shared static code metric data to conduct the experiment, since there are a number of "?" values. When we use SourceMonitor and Metrics plug-in (the two metric collection tools used by McBurney et al.) to collect the static code metric data, we do find that many "?" values are

Table 3
Performance comparison under the top 5% (33/664) on the Ant project.

Actual important classes	#Ranking by PageRank	PageRank	#Dependent on by others	#Ranking by QNC	QNC	#All dependencies and coupling	#Ranking by ADC	ADC
Project.java	1	✓	203	4	✓	1130	1	✓
Task.java	5	✓	256	2	✓	963	2	✓
Target.java	7	✓	19	20	✓	168	79	
ProjectHelper2.java	8	✓	4	142		93	165	
RuntimeConfigurable.java	13	✓	7	71		181	69	
UnknownElement.java	20	✓	11	42		259	34	
IntrospectionHelper.java	32	✓	5	85		281	27	✓
ProjectHelperImpl.java	416		0	541		76	206	

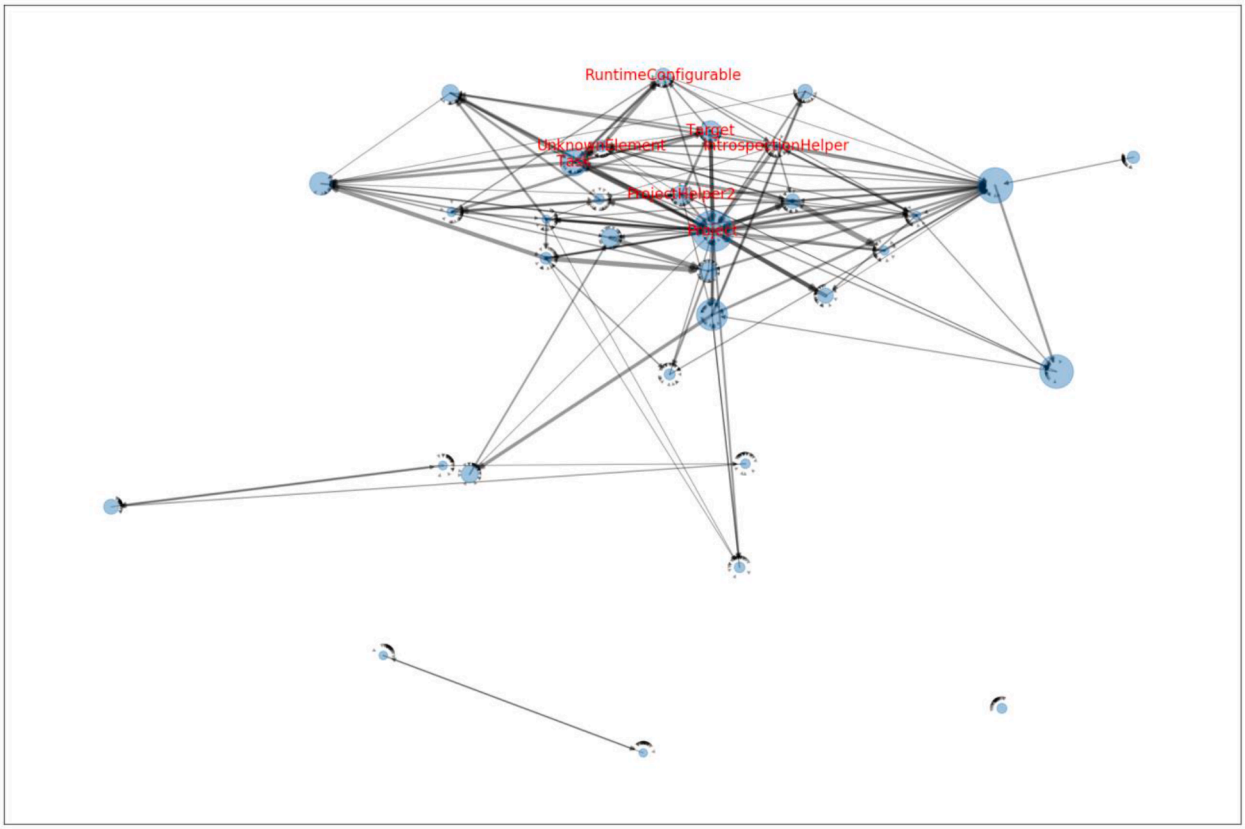

Fig. 6. The module dependence graph of the Ant project.

generated. This means that the "?" values in McBurney et al.'s shared data sets are caused by their used metric data collection tools (in their study, McBurney et al. did not report how to deal with these "?" values). Therefore, in our study, we use the script developed in our previous studies [44,53], which has been sufficiently tested, to collect the metric data. As for tf/idf metrics, McBurney et al. did not share the tf/idf metric data for the old data sets and the corresponding data collection tool. In our study, following in the descript in [16], we develop a script based on Weka to collect tf/idf metrics for all the nine projects under study. As for the PageRank scores, we developed our scripts to collect the data, which had been sufficiently tested to ensure its reliability. In this sense, the construct validity of the independent variable should be acceptable in our study.

7.2. Internal validity

Internal validity is the degree to which conclusions can be drawn about the causal effect of independent variables on the dependent variable. In our study, there are three possible threats to the internal validity.

The first threat comes from model validation technique. In our study, the model validation technique is 100-times out-of-sample bootstrap, different from a 10-fold cross validation used by McBurney et al. [16]. There are two reasons for making this choice. First, we hope to reduce the deviation of the experimental results from the model validation technique itself. Therefore, we use the out-of-sample bootstrap, as it is the least biased model validation technique [45]. Second, it is unclear how to compute the performance of an ANN model when applying a 10-fold cross validation to the NanoXML data set. As can be seen, the NanoXML data set consists of 21 instances (i.e. Java classes), of which only 6 are labeled as "important". If a 10-fold cross validation is applied, at least four folds will contain no "important" class. In this case, it is unclear how to compute the performance indicators (such as recall and precision) on these four folds (as no detail is given in McBurney et al.'s study). To reduce the influence of the initial order of instances in a data

set, we run 100 times out-of-sample bootstrap to evaluate the performance. In this way, we can objectively compare the effectiveness of the PageRank and ANN approaches[8]. Therefore, the threat from model validation technique has been considered as possible as we can.

The second threat is the unknown effect of the deviation of the independent variables from the normal distribution. In our study, we used the raw data to build the ANN models when investigating RQ1 and RQ2. In other words, we did not take into account whether the independent variables follow a normal distribution. The reason is that, in ANN, there is no assumption related to normal distribution. However, previous studies suggested applying the log transformation to the independent variables to make them close to a normal distribution, as it might lead to a better model [54]. To reduce this threat, we applied the log transformation and rerun the analyses. We found that the conclusions for RQ1 and RQ2 did not change before and after the log transformation.

The third threat is the unknown effect of feature selection method. In our study, we use CFS as the feature select method when building the ANN models. To reduce this threat, we used other representative feature selection methods such as ReliefF, InfoGain, and GainRatio [55] to re-ran the analysis and found the results to be very similar.

7.3. External validity

External validity is the degree to which the results of the research can be generalized to the population under study and other research settings. The most important threat is that our finding may not be generalized to non-Java projects or commercial projects. All projects in both the old data sets [16] and new data sets [37] used in this paper are open-source Java projects. To mitigate this threat, there is a need to replicate our study using a wider variety of projects in the future work.

8. Conclusion and future work

The goal of prioritizing code documentation effort is to identify modules that are important to program comprehension, and the documentation effort of these modules should take precedence. In this paper, we propose an unsupervised PageRank approach to prioritizing documentation effort. This approach identifies important modules only based on the dependence relationships between modules in a project. As a result, the PageRank approach does not need any training data to build the prediction model. Based on our experimental results on six new added large data sets and three data sets used in previous studies [16], we find that the PageRank approach is superior to the-state-of-the-art supervised ANN approach. Due to the simplicity and effectiveness, we strongly recommend that the PageRank approach should be used as an easy-to-implement baseline in future research, and that any new approach should be compared with the PageRank approach to demonstrate its effectiveness.

In future work, we plan to do more empirical studies on non-Java or closed-source projects to evaluate the effectiveness of the PageRank approach. In addition, we will try more strategies to further improve the PageRank approach.

CRediT authorship contribution statement

Shiran Liu: Conceptualization, Methodology, Software, Data curation, Writing – original draft. **Zhaoqiang Guo:** Software, Data curation, Investigation. **Yanhui Li:** Supervision, Investigation, Validation, Writing – review & editing. **Hongmin Lu:** Data curation, Investigation, Validation. **Lin Chen:** Investigation, Validation. **Lei Xu:** Investigation. **Yuming Zhou:** Supervision, Conceptualization, Methodology, Data curation, Validation, Writing – review & editing. **Baowen Xu:** Investigation.

[8] Note that, in this study, we aim to conduct an objective comparison between the PageRank and ANN approaches, rather than replicate McBurney et al.'s experimental results exactly. In fact, it is hard, if not possible, to exactly replicate their results, due to many details are missing in their paper.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work is partially supported by the National Natural Science Foundation of China (61772259, 61872177).

Supplementary materials

Supplementary material associated with this article can be found, in the online version, at doi:10.1016/j.infsof.2021.106686.

References

[1] A. Forward, T. Lethbridge, The relevance of software documentation, tools and technologies: a survey, in: 2002 ACM Symposium on Document Engineering, 2002, pp. 26–33. DocEng.
[2] T. Lethbridge, J. Singer, A. Forward, How software engineers use documentation: The state of the practice, IEEE Softw. 20 (6) (2003) 35–39.
[3] T. Roehm, R. Tiarks, R. Koschke, W. Maalej, How do professional developers comprehend software?, in: IEEE/ACM International Conference on Software Engineering, 2012, pp. 255–265. ICSE.
[4] S. Bugde, N. Nagappan, S.K. Rajamani, G. Ramalingam, Global software servicing: observational experiences at Microsoft, in: IEEE International Conference on Global Software Engineering, 2008, pp. 182–191. ICGSE.
[5] B. Fluri, M. Würsch, H.C. Gall, Do code and comments co-evolve? On the relation between source code and comment changes, in: 14th Working Conference on Reverse Engineering, 2007, pp. 70–79. WCRE.
[6] M. Kajko-Mattsson, A survey of documentation practice within corrective maintenance, Empir. Softw. Eng. 10 (1) (2005) 31–55.
[7] J. Sillito, G.C. Murphy, K.D. Volder, Asking and answering questions during a programming change task, IEEE Trans. Softw. Eng. 34 (4) (2008) 434–451.
[8] Y. Shmerlin, I. Hadar, D. Kliger, H. Makabee, To document or not to document? An exploratory study on developers' motivation to document code, in: 27th International Conference on Advanced Information Systems Engineering Workshops, CAiSE Workshops, 2015, pp. 100–106.
[9] P.W. McBurney, C. McMillan, Automatic documentation generation via source code summarization of method context, in: International Conference on Program Comprehension, 2014, pp. 279–290. ICPC.
[10] A.D. Lucia, M.D. Penta, R. Oliveto, A. Panichella, S. Panichella, Using IR methods for labeling source code artifacts: is it worthwhile?, in: International Conference on Program Comprehension, 2012, pp. 193–202. ICPC.
[11] P. Rodeghero, C. McMillan, P.W. McBurney, N. Bosch, S.K. D'Mello, Improving automated source code summarization via an eye-tracking study of programmers, in: IEEE/ACM International Conference on Software Engineering, 2014, pp. 390–401. ICSE.
[12] L. Moreno, J. Aponte, G. Sridhara, A. Marcus, L.L. Pollock, K. Vijay-Shanker, Automatic generation of natural language summaries for Java classes, in: International Conference on Program Comprehension, 2013, pp. 23–32. ICPC.
[13] L. Moreno, A. Marcus, L.L. Pollock, K. Vijay-Shanker, JSummarizer: An automatic generator of natural language summaries for Java classes, in: International Conference on Program Comprehension, 2013, pp. 230–232. ICPC.
[14] P. Rodeghero, S. Jiang, A. Armaly, C. McMillan, Detecting user story information in developer-client conversations to generate extractive summaries, in: IEEE/ACM International Conference on Software Engineering, 2017, pp. 49–59. ICSE.
[15] P.W. McBurney, C. McMillan, Automatic source code summarization of context for Java methods, IEEE Trans. Softw. Eng. 42 (2) (2016) 103–119.
[16] P.W. McBurney, S. Jiang, M. Kessentini, N.A. Kraft, A. Armaly, M.W. Mkaouer, C. McMillan, Towards prioritizing documentation effort, IEEE Trans. Softw. Eng. 44 (9) (2018) 897–913.
[17] G. Salton, A. Wong, C. Yang, A vector space model for automatic indexing, Commun. ACM 18 (11) (1975) 613–620.
[18] M.T. Hagan, H.B. Demuth, M. Beale, Neural Network Design, PWS Publishing, Boston, 1996.
[19] F. Shull, J.C. Carver, S. Vegas, N.J. Juzgado, The role of replications in empirical software engineering, Empir. Softw. Eng. 13 (2) (2008) 211–218.
[20] M.J. Shepperd, N. Ajienka, S. Counsell, The role and value of replication in empirical software engineering results, Inf. Softw. Technol. 99 (2018) 120–132.
[21] M. Cruz, B. Bernárdez, A. Durán, J.A. Galindo, A. Ruiz-Cortés, Replication of studies in empirical software engineering: a systematic mapping study, From 2013 to 2018, IEEE Access 8 (2020) 26773–26791.

[22] L. Page, S. Brin, R. Motwani, T. Winograd, The PageRank Citation Ranking: Bringing Order to the Web, Stanford InfoLab, 1999.
[23] B.J. Bauer, D.L. Parnas, Applying mathematical software documentation: an experience report, in: 10th Annual Conference on Computer Assurance, 1995, pp. 273–284. COMPASS.
[24] J. Kotula, Source code documentation: an engineering deliverable, in: 34th Technology of Object-Oriented Languages and Systems, 2000, p. 505. TOOLS.
[25] M. Visconti, C.R. Cook, An overview of industrial software documentation practice, in: 22nd International Conference of the Chilean Computer Science Society, 2002, pp. 179–186. SCCC.
[26] L.C. Briand, Software documentation: How much is enough?, in: 17th European Conference on Software Maintenance and Reengineering, 2003, p. 13. CSMR.
[27] M. Kajko-Mattsson, A survey of documentation practice within corrective maintenance, Empir. Softw. Eng. 10 (1) (2005) 31–55.
[28] M.A. Oumaziz, A. Charpentier, J. Falleri, X. Blanc, Documentation reuse: hot or not? An empirical study, in: International Conference on Software Reuse, 2017, pp. 12–27. ICSR.
[29] U. Dekel, J.D. Herbsleb, Reading the documentation of invoked API functions in program comprehension, in: International Conference on Program Comprehension, 2009, pp. 168–177. ICPC.
[30] D.K. Peters, D.L. Parnas, Generating a test oracle from program documentation (work in progress), in: International Symposium on Software Testing and Analysis, 1994, pp. 58–65. ISSTA.
[31] M.P. Robillard, R. DeLine, A field study of API learning obstacles, Empir. Softw. Eng. 16 (6) (2011) 703–732.
[32] K.C. Youm, J. Ahn, E. Lee, Improved bug localization based on code change histories and bug reports, Inf. Softw. Technol. 82 (2017) 177–192.
[33] Y. Zhou, R. Gu, T. Chen, Z. Huang, S. Panichella, H.C. Gall, Analyzing APIs documentation and code to detect directive defects, in: IEEE/ACM International Conference on Software Engineering, 2017, pp. 27–37. ICSE.
[34] Y. Xiong, J. Wang, R. Yan, J. Zhang, S. Han, G. Huang, L. Zhang, Precise condition synthesis for program repair, in: IEEE/ACM International Conference on Software Engineering, 2017, pp. 416–426. ICSE.
[35] Y. Zhou, B. Xu, Dependence structure analysis-based approach for measuring importance of classes, J. Southeast Univ. 38 (3) (2008) 380–384.
[36] H. Hu, J. Fang, Z. Lu, F. Zhao, Z. Qin, Rank-directed layout of UML class diagrams, Softw. Mining (2012) 25–31.
[37] I. Sora, Helping program comprehension of large software systems by identifying their most important classes, in: 10th International Conference on Evaluation of Novel Approaches to Software Engineering 599, Springer, Cham, 2015, pp. 122–140. ENASECommunications in Computer and Information Science.
[38] W. Pan, B. Song, K. Li, K. Zhang, Identifying key classes in object-oriented software using generalized k-core decomposition, Future Gener. Comput. Syst. 81 (2018) 188–202.
[39] M. Zhang, Y. Li, X. Li, L. Chen, Y. Zhang, L. Zhang, S. Khurshid, An empirical study of boosting spectrum-based fault localization via PageRank, IEEE Trans. Softw. Eng. 47 (6) (2021) 1089–1113.
[40] C. Zhang, H. Jacobsen, Mining crosscutting concerns through random walks, IEEE Trans. Softw. Eng. 38 (5) (2012) 1123–1137.
[41] G. Kumar, N. Duhan, A.K Sharma, Page ranking based on number of visits of links of web page, in: 2nd International Conference on Computer and Communication Technology, 2011, pp. 11–14. ICCCT.
[42] R. Mihalcea, P. Tarau, TextRank: bringing order into text, in: Conference on Empirical Methods in Natural Language Processing, 2004, pp. 404–411. EMNLP.
[43] M.A. Hall, L.A. Smith, Feature selection for machine learning: Comparing a correlation-based filter approach to the wrapper, in: Twelfth International Florida Artificial Intelligence Research Society Conference, 1999, pp. 235–239. FLAIRS.
[44] Y. Zhou, B. Xu, H. Leung, L. Chen, An in-depth study of the potentially confounding effect of class size in fault prediction, ACM Trans. Softw. Eng. Methodol. 23 (1) (2014) 1–51.
[45] T. Hall, S. Beecham, D. Bowes, D. Gray, S. Counsell, A systematic literature review on fault prediction performance in software engineering, IEEE Trans. Softw. Eng. 38 (6) (2012) 1276–1304.
[46] B. Efron, Estimating the error rate of a prediction rule: improvement on cross-validation, J. Am. Stat. Assoc. 78 (382) (1983) 316–331.
[47] C. Tantithamthavorn, S. McIntosh, A.E. Hassan, K. Matsumoto, An empirical comparison of model validation techniques for defect prediction models, IEEE Trans. Softw. Eng. 43 (1) (2017) 1–18.
[48] M. Hollander, D.A. Wolfe, Nonparametric Statistical Methods, John Wiley & Sons, New York, 1973.
[49] N. Cliff, Ordinal Methods for Behavioral Data Analysis, Routledge, 1996.
[50] Y. Benjamini, Y. Hochberg, Controlling the false discovery rate: a practical and powerful approach to multiple testing, J. R. Stat. Soc. Ser. B 57 (1995) 289–300.
[51] Y. Zhou, Y. Yang, H. Lu, L. Chen, Y. Li, Y. Zhao, J. Qian, B. Xu, How far we have progressed in the journey? An examination of cross-project defect prediction, ACM Trans. Softw. Eng. Methodol. 27 (1) (2018) 1–51, article 1.
[52] Q. Song, Y. Guo, M.J. Shepperd, A comprehensive investigation of the role of imbalanced learning for software defect prediction, IEEE Trans. Softw. Eng. 45 (12) (2019) 1253–1269.
[53] Y. Zhou, H.K.N Leung, B. Xu, Examining the potentially confounding effect of class size on the associations between object-oriented metrics and change-proneness, IEEE Trans. Softw. Eng. 35 (5) (2009) 607–623.
[54] T. Menzies, J. Greenwald, A. Frank, Data mining static code attributes to learn defect predictors, IEEE Trans. Softw. Eng. 33 (1) (2007) 2–13.
[55] B. Ghotra, S. McIntosh, A.E. Hassan, A large-scale study of the impact of feature selection techniques on defect classification models, in: 14th International Conference on Mining Software Repositories, 2017, pp. 146–157. MSR.
[56] N. Peitek, S. Apel, C. Parnin, Program comprehension and code complexity metrics: an fMRI study, in: IEEE/ACM International Conference on Software Engineering, ICSE, 2021 accepted.
[57] N. Wilde, Understanding Program Dependencies, SEI-CM-26, 1990.
[58] A.J. Ko, B.A. Myers, M.J. Coblenz, H.H. Aung, An exploratory study of how developers seek, relate, and collect relevant information during software maintenance tasks, IEEE Trans. Softw. Eng. 32 (12) (2006) 971–987.
[59] M. BianChini, M. Gori, F. Scarselli, Inside PageRank, ACM Trans. Internet Technol. 5 (1) (2005) 92–128.
[60] A.N. Langville, C.D. Meyer, Deeper inside PageRank, Internet Math. 1 (3) (2003) 335–380.
[61] D.F. Gleich, PageRank beyond the Web, SIAM Rev. 57 (3) (2015) 321–363.